S

BỘ NÔNG NGHIỆP VÀ MÔI TRƯỜNG

PHÂN HIỆU TRƯỜNG ĐẠI HỌC THỦY LỢI

BỘ MÔN CÔNG NGHỆ THÔNG TIN

BÁO CÁO MÔN

CHUYÊN ĐỀ CÔNG NGHỆ THÔNG TIN

Tên đề tài:

XÂY DỰNG AGENTIC - CHATBOT HỖ TRỢ THAM QUAN ĐỊA ĐẠO CỦ CHI

Giảng viên hướng dẫn:	**TS.Hoàng Văn Quý**
Sinh viên thực hiện:	***Nguyễn Thị Yến Ngọc***
MSSV:	***2351067103***
Lớp:	***S26-65CNTT***

2026

Lời Cám Ơn

Để hoàn thành được tiểu luận này, em xin chân thành cảm ơn Ban Giám hiệu, các khoa, phòng và quý thầy, cô của trường Phân hiệu Đại học Thuỷ Lợi, những người đã tận tình giúp đỡ và tạo điều kiện cho em trong quá trình học tập. Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến thầy Hoàng Văn Quý - người đã trực tiếp giảng dạy và hướng dẫn em thực hiện bài tiểu luận này bằng tất cả lòng nhiệt tình và sự quan tâm sâu sắc.

Trong quá trình thực hiện đồ án này, do hiểu biết còn nhiều hạn chế nên bài làm khó tránh khỏi những thiếu sót. Em rất mong nhận được những lời góp ý của quý thầy cô để bài tiểu luận ngày càng hoàn thiện hơn.

Em xin chân thành cảm ơn!

MỤC LỤC

LỜI MỞ ĐẦU 7

1. Lý do chọn đề tài 7

2. Mục tiêu đề tài 8

3. Đối tượng và phạm vi nghiên cứu 8

3.1. Đối tượng nghiên cứu 9

3.2. Phạm vi nghiên cứu 9

4. Phương pháp thực hiện và kết quả dự kiến 9

4.1. Phương pháp thực hiện 9

4.2. Kết quả dự kiến 10

CHƯƠNG I. CƠ SỞ LÝ THUYẾT 12

1.1. Tổng quan về chatbot và trí tuệ nhân tạo 12

1.1.1. Khái niệm trí tuệ nhân tạo 12

1.1.2. Khái niệm chatbot 12

1.1.3. Ứng dụng chatbot trong hỗ trợ tham quan du lịch 13

1.2. Tổng quan về Agentic AI 13

1.2.1. Khái niệm Agentic AI 13

1.2.2. Các thành phần cơ bản của hệ thống Agentic 13

1.2.3. Phân biệt chatbot truyền thống và Agentic chatbot 14

1.3. Mô hình ngôn ngữ lớn (Large Language Model – LLM) 14

1.3.1. Khái niệm mô hình ngôn ngữ lớn 14

1.3.2. Vai trò của LLM trong hệ thống chatbot 15

1.3.3. Giới thiệu mô hình Qwen2.5-3B-Instruct 15

1.3.4. Lý do lựa chọn mô hình LLM cục bộ 15

1.4. Kỹ thuật Retrieval-Augmented Generation (RAG) 16

1.4.1. Khái niệm RAG 16

1.4.2. Khái niệm Embedding 16

1.4.3. Tìm kiếm ngữ nghĩa và độ tương đồng vector 17

1.4.4. Giới thiệu thư viện FAISS 17

1.4.5. Vai trò của RAG trong đề tài 17

1.5. Công nghệ và công cụ sử dụng 18

1.5.1. Ngôn ngữ lập trình Python 18

1.5.2. Framework FastAPI 18

1.5.3. Hệ quản trị cơ sở dữ liệu PostgreSQL 18
1.5.4. Thư viện PyTorch và Transformers 19
1.5.5. Sentence Transformers và FAISS 19
1.5.6. HTML, CSS và JavaScript 19
CHƯƠNG II. PHÂN TÍCH VÀ XÂY DỰNG HỆ THỐNG AGENTIC CHATBOT HỖ TRỢ THAM QUAN ĐỊA ĐẠO CỦ CHI 20
2.1. Phân tích yêu cầu hệ thống 20
2.1.1. Mô tả bài toán 20
2.1.2. Yêu cầu chức năng 20
2.1.3. Yêu cầu phi chức năng 21
2.2. Phân tích tác nhân và chức năng hệ thống 21
2.2.1. Xác định tác nhân 21
2.2.2. Các trường hợp sử dụng 22
2.2.3. Mô tả quy trình sử dụng 22
2.3. Xây dựng kiến trúc tổng thể của hệ thống 22
2.3.1. Mô hình kiến trúc 22
2.3.2. Sự phối hợp giữa các thành phần 23
2.3.3. Đặc điểm kiến trúc 24
2.4. Xây dựng cơ sở dữ liệu PostgreSQL 24
2.4.1. Mục đích sử dụng cơ sở dữ liệu 24
2.4.2. Thực thể phiên trò chuyện 24
2.4.3. Thực thể tin nhắn 25
2.4.4. Mối quan hệ giữa các thực thể 27
2.4.5. Vai trò của PostgreSQL trong hệ thống 27
2.5. Xây dựng bộ dữ liệu và hệ thống RAG 27
2.5.1. Xây dựng bộ dữ liệu kiến thức 27
2.5.2. Tiền xử lý và tạo embedding 28
2.5.3. Xây dựng chỉ mục FAISS 28
2.5.4. Quy trình truy xuất tài liệu 28
2.5.5. Vai trò của RAG đối với chất lượng câu trả lời 29
2.6. Xây dựng Agent và cơ chế điều phối công cụ 29
2.6.1. Vai trò của Agent 29
2.6.2. Phân loại ý định người dùng 29
2.6.3. Các công cụ của Agent 30

2.6.4. Cơ chế điều phối xử lý 30
2.6.5. Ghi nhận Agent Trace 30
2.7. Luồng xử lý câu hỏi và sinh câu trả lời 31
2.7.1. Các bước xử lý câu hỏi 31
2.7.2. Xử lý các yêu cầu không cần truy xuất RAG 33
2.8. Thiết kế giao diện và tích hợp hệ thống 33
2.8.1. Giao diện chatbot 34
2.8.2. Hiển thị trạng thái xử lý 34
2.8.3. Hiển thị nguồn tham khảo và Agent Trace 34
2.8.4. Tích hợp frontend và backend 35
2.8.5. Đánh giá quá trình tích hợp 35
CHƯƠNG III. KẾT QUẢ THỰC NGHIỆM 37
3.1. Kết quả hoạt động của hệ thống 37
3.2. Đánh giá kết quả đạt được 38
3.3. Kết luận 38
3.4. Hướng phát triển 39
Tài liệu tham khảo 40

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TPHCM, ngày 05 tháng 11 năm 2026

Ký và ghi rõ họ tên

Hoàng Văn Quý

LỜI MỞ ĐẦU

1. Lý do chọn đề tài

Trong những năm gần đây, sự phát triển của công nghệ thông tin và trí tuệ nhân tạo (Artificial Intelligence – AI) đã tạo ra nhiều cơ hội ứng dụng trong các lĩnh vực của đời sống, đặc biệt là du lịch, giáo dục và cung cấp thông tin. Các hệ thống chatbot ứng dụng trí tuệ nhân tạo ngày càng được quan tâm nhờ khả năng tương tác với người dùng bằng ngôn ngữ tự nhiên, hỗ trợ giải đáp thắc mắc và cung cấp thông tin một cách thuận tiện.

Địa đạo Củ Chi là một di tích lịch sử nổi tiếng tại Thành phố Hồ Chí Minh, gắn liền với quá trình đấu tranh của quân và dân Việt Nam trong thời kỳ kháng chiến. Đây là địa điểm tham quan, tìm hiểu lịch sử thu hút nhiều du khách trong nước và quốc tế. Khi có nhu cầu tham quan Địa đạo Củ Chi, du khách thường tìm kiếm các thông tin như lịch sử hình thành, những khu vực tham quan, giá vé, thời gian hoạt động, phương tiện di chuyển và các lưu ý trong quá trình tham quan.

Hiện nay, những thông tin này có thể được tìm thấy thông qua các trang web, công cụ tìm kiếm hoặc tài liệu giới thiệu. Tuy nhiên, người dùng thường phải tự tìm kiếm, lựa chọn và tổng hợp thông tin từ nhiều nguồn khác nhau. Điều này có thể gây mất thời gian, đặc biệt đối với những người chưa quen thuộc với địa điểm tham quan hoặc muốn được giải đáp nhanh một câu hỏi cụ thể.

Sự phát triển của các mô hình ngôn ngữ lớn (Large Language Model – LLM) mở ra khả năng xây dựng những hệ thống hỏi đáp có thể hiểu và tạo phản hồi bằng ngôn ngữ tự nhiên. Bên cạnh đó, kỹ thuật Retrieval-Augmented Generation (RAG) cho phép kết hợp khả năng sinh văn bản của mô hình ngôn ngữ với việc truy xuất thông tin từ một bộ dữ liệu được xây dựng trước, giúp câu trả lời có cơ sở tham khảo phù hợp hơn. Việc kết hợp cơ chế Agentic AI còn hỗ trợ tổ chức quy trình xử lý câu hỏi và điều phối các công cụ trong hệ thống.

Xuất phát từ nhu cầu thực tế và mong muốn tìm hiểu, ứng dụng các công nghệ trí tuệ nhân tạo vào một bài toán cụ thể, em lựa chọn đề tài **"Xây dựng Agentic – Chatbot hỗ trợ tham quan Địa đạo Củ Chi"**. Đề tài hướng đến việc xây dựng một ứng dụng chatbot trên nền tảng web, hỗ trợ người dùng tra cứu thông tin về Địa đạo Củ Chi bằng ngôn ngữ tự nhiên, đồng thời vận dụng các kiến thức về lập trình Python, cơ sở dữ liệu, mô hình ngôn ngữ lớn, RAG và cơ chế điều phối Agent.

2. Mục tiêu đề tài

Mục tiêu chính của đề tài là xây dựng một hệ thống chatbot hỗ trợ người dùng tìm hiểu và tra cứu thông tin liên quan đến hoạt động tham quan Địa đạo Củ Chi thông qua giao diện web. Hệ thống sử dụng mô hình ngôn ngữ lớn triển khai cục bộ, kết hợp kỹ thuật RAG và cơ chế Agentic để tiếp nhận, xử lý câu hỏi và tạo câu trả lời phù hợp với dữ liệu đã xây dựng.

Để đạt được mục tiêu trên, đề tài tập trung thực hiện các mục tiêu cụ thể sau:

- *Xây dựng bộ dữ liệu kiến thức*: Thu thập, tổng hợp và tổ chức các thông tin liên quan đến Địa đạo Củ Chi, bao gồm kiến thức giới thiệu, lịch sử, địa điểm tham quan, giá vé, phương tiện di chuyển và các câu hỏi thường gặp.
- *Xây dựng hệ thống truy xuất thông tin RAG*: Sử dụng mô hình embedding để chuyển đổi dữ liệu văn bản thành vector, lưu trữ trong FAISS và tìm kiếm các tài liệu có nội dung liên quan đến câu hỏi của người dùng.
- *Tích hợp mô hình ngôn ngữ lớn cục bộ*: Sử dụng mô hình Qwen2.5-3B-Instruct để tạo câu trả lời bằng ngôn ngữ tự nhiên dựa trên câu hỏi và thông tin được truy xuất, không phụ thuộc vào dịch vụ API suy luận AI bên ngoài.
- *Xây dựng cơ chế điều phối Agentic*: Thiết kế thành phần Agent có khả năng phân loại ý định theo quy tắc, lựa chọn và phối hợp các công cụ tìm kiếm kiến thức, tìm kiếm câu hỏi thường gặp và khai thác lịch sử hội thoại.
- *Phát triển ứng dụng chatbot trên nền tảng web*: Xây dựng giao diện cho phép người dùng nhập câu hỏi, xem câu trả lời, nguồn tham khảo và thông tin các bước xử lý của Agent.
- *Quản lý dữ liệu hội thoại*: Sử dụng hệ quản trị cơ sở dữ liệu PostgreSQL để lưu trữ phiên trò chuyện và lịch sử trao đổi giữa người dùng với chatbot.
- *Kiểm thử và đánh giá hệ thống*: Thực hiện các kịch bản thử nghiệm nhằm kiểm tra khả năng hoạt động của các thành phần, đánh giá kết quả xử lý câu hỏi và xác định những hạn chế cần cải thiện.

Thông qua việc thực hiện các mục tiêu trên, đề tài giúp vận dụng kiến thức đã học vào quá trình phân tích, thiết kế, lập trình, tích hợp và kiểm thử một hệ thống phần mềm có ứng dụng trí tuệ nhân tạo.

3. Đối tượng và phạm vi nghiên cứu

3.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là hệ thống chatbot ứng dụng trí tuệ nhân tạo nhằm hỗ trợ người dùng tra cứu thông tin tham quan Địa đạo Củ Chi.

Các nội dung nghiên cứu tập trung vào mô hình ngôn ngữ lớn, kỹ thuật truy xuất tăng cường sinh văn bản RAG, cơ chế điều phối Agentic, phương pháp biểu diễn văn bản bằng vector và các công nghệ phục vụ xây dựng ứng dụng web.

Bên cạnh đó, đề tài nghiên cứu cách tổ chức dữ liệu kiến thức, thiết kế cơ sở dữ liệu lưu trữ hội thoại và xây dựng quy trình phối hợp giữa các thành phần để xử lý yêu cầu của người dùng.

3.2. Phạm vi nghiên cứu

Về nội dung, hệ thống tập trung hỗ trợ giải đáp các câu hỏi liên quan đến Địa đạo Củ Chi, bao gồm thông tin giới thiệu, lịch sử, các khu vực tham quan, giá vé, thời gian hoạt động, đường đi, phương tiện di chuyển và những lưu ý dành cho du khách.

Về chức năng, hệ thống được xây dựng dưới dạng ứng dụng web với các chức năng chính như tiếp nhận câu hỏi, truy xuất thông tin, tạo câu trả lời, hiển thị nguồn tham khảo, tạo phiên trò chuyện và lưu trữ lịch sử hội thoại. Hệ thống hướng đến khả năng tiếp nhận và phản hồi các câu hỏi bằng tiếng Việt và tiếng Anh trong phạm vi dữ liệu được cung cấp.

Về công nghệ, đề tài sử dụng ngôn ngữ lập trình Python, framework FastAPI để xây dựng backend, PostgreSQL để quản lý dữ liệu hội thoại, FAISS để lưu trữ và tìm kiếm vector, mô hình embedding đa ngôn ngữ để biểu diễn văn bản và mô hình Qwen2.5-3B-Instruct để sinh câu trả lời. Giao diện được phát triển bằng HTML, CSS và JavaScript.

Về môi trường triển khai, hệ thống được xây dựng và thử nghiệm trên máy tính cá nhân sử dụng hệ điều hành Windows. Các mô hình AI được triển khai cục bộ, phù hợp với mục tiêu xây dựng một sản phẩm thử nghiệm phục vụ học tập và nghiên cứu.

Đề tài tập trung vào việc xây dựng và kiểm chứng một hệ thống chatbot có khả năng hoạt động, chưa hướng đến triển khai thương mại hoặc phục vụ số lượng lớn người dùng đồng thời. Thông tin được cung cấp mang tính chất hỗ trợ tham khảo; một số dữ liệu có thể thay đổi theo thời gian và cần được đối chiếu với nguồn chính thức trước khi sử dụng.

4. Phương pháp thực hiện và kết quả dự kiến

4.1. Phương pháp thực hiện

Để thực hiện đề tài, các phương pháp nghiên cứu và triển khai được áp dụng như sau:

Phương pháp nghiên cứu tài liệu: Tìm hiểu các kiến thức liên quan đến chatbot, Agentic AI, mô hình ngôn ngữ lớn, kỹ thuật RAG, mô hình embedding và cơ sở dữ liệu vector. Đồng thời, khảo sát các công nghệ và hệ thống có liên quan để lựa chọn hướng xây dựng phù hợp với yêu cầu của đề tài.

Phương pháp thu thập và tổ chức dữ liệu: Tổng hợp các thông tin liên quan đến Địa đạo Củ Chi, tổ chức dữ liệu thành các nhóm kiến thức và câu hỏi thường gặp. Dữ liệu được lưu trữ dưới dạng các tệp có cấu trúc để phục vụ quá trình xử lý và xây dựng chỉ mục tìm kiếm.

Phương pháp phân tích và thiết kế hệ thống: Xác định các yêu cầu chức năng, thiết kế kiến trúc phần mềm, mô hình cơ sở dữ liệu và quy trình xử lý câu hỏi. Phân chia hệ thống thành các thành phần riêng biệt gồm giao diện người dùng, backend, Agent, RAG, mô hình ngôn ngữ và cơ sở dữ liệu.

Phương pháp xây dựng và tích hợp: Phát triển các thành phần bằng Python và FastAPI; xây dựng chỉ mục FAISS từ dữ liệu kiến thức; tích hợp mô hình Qwen2.5-3B-Instruct chạy cục bộ; xây dựng cơ chế điều phối Agent và kết nối với PostgreSQL để lưu trữ lịch sử hội thoại.

Phương pháp kiểm thử và đánh giá: Thử nghiệm các chức năng của hệ thống thông qua những câu hỏi liên quan đến nội dung tham quan Địa đạo Củ Chi. Quan sát kết quả truy xuất dữ liệu, câu trả lời của chatbot, hoạt động của các công cụ và thời gian xử lý nhằm đánh giá mức độ đáp ứng mục tiêu đề tài.

4.2. Kết quả dự kiến

Sau quá trình nghiên cứu và triển khai, đề tài hướng đến các kết quả sau:

- Xây dựng được ứng dụng chatbot trên nền tảng web, có giao diện cho phép người dùng đặt câu hỏi và nhận câu trả lời bằng ngôn ngữ tự nhiên.
- Xây dựng được bộ dữ liệu kiến thức về Địa đạo Củ Chi, kết hợp với hệ thống truy xuất RAG sử dụng mô hình embedding và FAISS.
- Tích hợp được mô hình Qwen2.5-3B-Instruct hoạt động cục bộ để tạo câu trả lời dựa trên dữ liệu tham khảo.

- Xây dựng được thành phần Agent điều phối các công cụ xử lý câu hỏi, truy xuất kiến thức và khai thác lịch sử hội thoại.
- Sử dụng PostgreSQL để lưu trữ phiên trò chuyện và nội dung trao đổi giữa người dùng với chatbot.
- Hiển thị được nguồn tham khảo và thông tin các bước xử lý, hỗ trợ quan sát hoạt động của hệ thống.
- Thực hiện được các thử nghiệm cơ bản, đánh giá khả năng hoạt động và xác định những hạn chế về hiệu năng, dữ liệu và phạm vi ứng dụng.

CHƯƠNG I. CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về chatbot và trí tuệ nhân tạo

1.1.1. Khái niệm trí tuệ nhân tạo

Trí tuệ nhân tạo (Artificial Intelligence – AI) là lĩnh vực nghiên cứu và phát triển các hệ thống máy tính có khả năng thực hiện những nhiệm vụ thường đòi hỏi trí thông minh của con người, chẳng hạn như nhận dạng thông tin, xử lý ngôn ngữ, suy luận và hỗ trợ ra quyết định.

Trong những năm gần đây, trí tuệ nhân tạo được ứng dụng rộng rãi trong nhiều lĩnh vực như giáo dục, y tế, thương mại điện tử, dịch vụ khách hàng và du lịch. Đặc biệt, sự phát triển của các mô hình học sâu đã tạo điều kiện xây dựng những ứng dụng có khả năng tiếp nhận và xử lý thông tin dưới dạng ngôn ngữ tự nhiên.

Đối với lĩnh vực du lịch, AI có thể được ứng dụng để hỗ trợ tìm kiếm địa điểm, giới thiệu điểm tham quan, giải đáp các câu hỏi thường gặp và cung cấp thông tin phù hợp với nhu cầu của người dùng.

1.1.2. Khái niệm chatbot

Chatbot là chương trình phần mềm được thiết kế nhằm mô phỏng hoạt động hội thoại giữa con người và máy tính thông qua văn bản hoặc giọng nói. Người dùng có thể đặt câu hỏi bằng ngôn ngữ tự nhiên và nhận phản hồi từ hệ thống.

Dựa trên phương pháp xử lý, chatbot có thể được chia thành một số nhóm chính:

Chatbot dựa trên quy tắc (Rule-based Chatbot): Hoạt động dựa trên các điều kiện, từ khóa hoặc kịch bản hội thoại được xây dựng trước. Loại chatbot này tương đối dễ triển khai, có khả năng kiểm soát phản hồi nhưng thường gặp khó khăn khi người dùng đặt câu hỏi ngoài những mẫu đã định nghĩa.

*Chatbot dựa trên mô hình học máy***:** Sử dụng các phương pháp học máy để nhận diện ý định hoặc lựa chọn phản hồi dựa trên dữ liệu huấn luyện. Khả năng xử lý có thể linh hoạt hơn chatbot dựa trên quy tắc nhưng phụ thuộc vào chất lượng dữ liệu và phương pháp huấn luyện.

*Chatbot sử dụng mô hình ngôn ngữ lớn***:** Tận dụng khả năng hiểu ngữ cảnh và sinh văn bản của LLM để tạo phản hồi bằng ngôn ngữ tự nhiên. Loại chatbot này có thể xử lý nhiều

cách diễn đạt khác nhau, tuy nhiên cần có biện pháp hạn chế việc tạo ra thông tin thiếu chính xác.

1.1.3. Ứng dụng chatbot trong hỗ trợ tham quan du lịch

Trong hoạt động du lịch, người dùng thường có nhu cầu tìm kiếm thông tin về địa điểm, lịch sử, thời gian hoạt động, giá vé, phương tiện di chuyển và những lưu ý khi tham quan. Chatbot có thể hỗ trợ quá trình này thông qua hình thức hỏi đáp trực tiếp, giúp người dùng tiếp cận thông tin thuận tiện hơn.

So với việc phải tìm kiếm và đọc nhiều trang thông tin, chatbot cho phép người dùng đặt câu hỏi cụ thể, chẳng hạn như "Đi xe buýt đến Địa đạo Củ Chi như thế nào?" hoặc "Địa đạo Bến Dược có gì để tham quan?". Hệ thống sẽ tiếp nhận câu hỏi, xác định nội dung cần tra cứu và tạo phản hồi phù hợp.

Tuy nhiên, hiệu quả của chatbot phụ thuộc vào dữ liệu được sử dụng, khả năng hiểu câu hỏi và phương pháp tạo câu trả lời. Đối với các thông tin có thể thay đổi như giá vé, giờ hoạt động hoặc số điện thoại liên hệ, việc kiểm tra và cập nhật dữ liệu là yêu cầu quan trọng.

1.2. Tổng quan về Agentic AI

1.2.1. Khái niệm Agentic AI

Agentic AI là hướng tiếp cận xây dựng hệ thống trí tuệ nhân tạo có khả năng thực hiện nhiệm vụ thông qua việc xác định các bước xử lý, sử dụng công cụ và phối hợp nhiều thành phần để đạt được mục tiêu.

Trong một hệ thống Agentic, thành phần Agent giữ vai trò điều phối quá trình xử lý. Tùy theo thiết kế, Agent có thể tiếp nhận yêu cầu, xác định ý định của người dùng, lựa chọn công cụ, thu thập thông tin và tổng hợp kết quả.

Không phải mọi hệ thống Agentic đều có mức độ tự chủ giống nhau. Một số hệ thống sử dụng mô hình ngôn ngữ để lập kế hoạch và lựa chọn công cụ một cách linh hoạt; một số hệ thống khác sử dụng các quy tắc lập trình để điều phối công cụ theo quy trình đã xác định.

1.2.2. Các thành phần cơ bản của hệ thống Agentic

Một hệ thống Agentic thường bao gồm những thành phần sau:

- Agent**:** Thành phần trung tâm tiếp nhận yêu cầu và tổ chức quá trình xử lý.

- Công cụ (Tool): Các chức năng hoặc dịch vụ mà Agent có thể sử dụng để thực hiện nhiệm vụ, chẳng hạn như tìm kiếm tài liệu, truy vấn cơ sở dữ liệu hoặc lấy lịch sử hội thoại.
- Ngữ cảnh (Context): Thông tin liên quan đến yêu cầu hiện tại, có thể bao gồm câu hỏi, lịch sử trò chuyện và kết quả từ các công cụ.
- Cơ chế điều phối (Orchestration): Quy trình xác định công cụ cần sử dụng, thứ tự thực hiện và cách tổng hợp kết quả.

Kết quả xử lý: Nội dung cuối cùng được tạo ra hoặc tổng hợp để phản hồi cho người dùng.

1.2.3. Phân biệt chatbot truyền thống và Agentic chatbot

Chatbot truyền thống thường xử lý yêu cầu theo một luồng tương đối cố định, chẳng hạn như nhận câu hỏi và trả về câu trả lời có sẵn.

Trong khi đó, Agentic chatbot có thể tổ chức nhiều bước xử lý và sử dụng các công cụ khác nhau để đáp ứng yêu cầu. Chẳng hạn, khi nhận được một câu hỏi về địa điểm tham quan, hệ thống có thể thực hiện tìm kiếm kiến thức, tìm kiếm câu hỏi thường gặp, tổng hợp tài liệu và chuyển thông tin cho mô hình ngôn ngữ tạo câu trả lời.

Đối với đề tài này, cơ chế Agentic được triển khai theo hướng điều phối dựa trên quy tắc. Thành phần Agent sử dụng logic Python để phân loại ý định và phối hợp các công cụ truy xuất dữ liệu. Hệ thống chưa triển khai cơ chế để LLM tự lập kế hoạch và tự quyết định gọi công cụ theo nhiều vòng suy luận độc lập.

Cách tiếp cận này phù hợp với phạm vi một đồ án thử nghiệm, giúp quy trình xử lý rõ ràng, dễ kiểm tra và thuận lợi cho việc phân tích hoạt động của hệ thống.

1.3. Mô hình ngôn ngữ lớn (Large Language Model – LLM)

1.3.1. Khái niệm mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn (Large Language Model – LLM) là mô hình trí tuệ nhân tạo được huấn luyện trên lượng dữ liệu văn bản lớn nhằm học các đặc điểm và quy luật của ngôn ngữ.

LLM có thể thực hiện nhiều nhiệm vụ như trả lời câu hỏi, tóm tắt văn bản, dịch thuật, diễn đạt lại nội dung và sinh văn bản theo yêu cầu. Phần lớn các mô hình ngôn ngữ hiện đại

sử dụng kiến trúc Transformer, trong đó cơ chế Attention giúp mô hình xử lý mối liên hệ giữa các thành phần trong chuỗi đầu vào.

Khi nhận một câu hỏi, mô hình sẽ xử lý chuỗi token đầu vào và lần lượt dự đoán các token tiếp theo để hình thành câu trả lời.

1.3.2. Vai trò của LLM trong hệ thống chatbot

Trong hệ thống chatbot, LLM có nhiệm vụ tạo phản hồi bằng ngôn ngữ tự nhiên. Thay vì chỉ lựa chọn một câu trả lời cố định, mô hình có thể diễn đạt thông tin theo nội dung câu hỏi và ngữ cảnh được cung cấp.

Tuy nhiên, LLM không phải cơ sở dữ liệu bảo đảm tính chính xác tuyệt đối. Mô hình có thể tạo ra thông tin không đúng hoặc không được hỗ trợ bởi nguồn dữ liệu, thường được gọi là hiện tượng hallucination.

Vì vậy, khi xây dựng chatbot phục vụ tra cứu kiến thức về một địa điểm cụ thể, việc kết hợp LLM với cơ chế truy xuất tài liệu là hướng tiếp cận phù hợp nhằm cung cấp thêm căn cứ cho quá trình sinh câu trả lời.

1.3.3. Giới thiệu mô hình Qwen2.5-3B-Instruct

Qwen2.5-3B-Instruct là mô hình ngôn ngữ thuộc họ Qwen2.5, được phát triển theo hướng hỗ trợ thực hiện các chỉ dẫn của người dùng. Mô hình có quy mô khoảng 3 tỷ tham số và hỗ trợ xử lý nhiều ngôn ngữ.

Trong đề tài, mô hình Qwen2.5-3B-Instruct được lựa chọn để thực hiện nhiệm vụ sinh câu trả lời dựa trên câu hỏi của người dùng và nội dung tài liệu do hệ thống RAG cung cấp.

Mô hình được lưu trữ và tải từ thư mục cục bộ, sử dụng thư viện Transformers và PyTorch để thực hiện suy luận. Cách triển khai này không yêu cầu gửi câu hỏi đến dịch vụ API suy luận AI bên ngoài.

Việc sử dụng mô hình cục bộ giúp chủ động trong quá trình triển khai và kiểm soát luồng xử lý dữ liệu. Tuy nhiên, hiệu năng phụ thuộc vào tài nguyên CPU, GPU và bộ nhớ của máy tính, đặc biệt khi mô hình có kích thước lớn so với cấu hình phần cứng.

1.3.4. Lý do lựa chọn mô hình LLM cục bộ

Việc lựa chọn mô hình ngôn ngữ triển khai cục bộ xuất phát từ những yêu cầu và điều kiện thực hiện của đề tài.

Thứ nhất, hệ thống cần có khả năng hoạt động mà không phụ thuộc vào dịch vụ API suy luận AI bên ngoài.

Thứ hai, việc triển khai cục bộ cho phép nghiên cứu trực tiếp quá trình tải mô hình, xây dựng prompt, thực hiện suy luận và đánh giá tốc độ sinh câu trả lời.

Thứ ba, mô hình Qwen2.5-3B-Instruct có quy mô phù hợp để thử nghiệm trên máy tính cá nhân, mặc dù tốc độ xử lý có thể còn hạn chế.

Từ những yếu tố trên, đề tài lựa chọn Qwen2.5-3B-Instruct làm thành phần sinh ngôn ngữ và kết hợp với RAG để khai thác dữ liệu kiến thức riêng về Địa đạo Củ Chi.

1.4. Kỹ thuật Retrieval-Augmented Generation (RAG)

1.4.1. Khái niệm RAG

Retrieval-Augmented Generation (RAG) là kỹ thuật kết hợp quá trình truy xuất thông tin với quá trình sinh văn bản bằng mô hình ngôn ngữ lớn.

Thay vì chỉ dựa vào kiến thức được học trong quá trình huấn luyện, hệ thống RAG tìm kiếm các tài liệu liên quan đến câu hỏi của người dùng, sau đó đưa nội dung truy xuất được vào ngữ cảnh đầu vào của LLM.

Quy trình RAG thường gồm hai giai đoạn chính:

- *Giai đoạn chuẩn bị dữ liệu*: Thu thập tài liệu, xử lý văn bản, tạo vector biểu diễn bằng mô hình embedding và lưu trữ các vector vào hệ thống tìm kiếm.
- *Giai đoạn trả lời câu hỏi*: Chuyển câu hỏi thành vector, tìm kiếm các tài liệu tương đồng, xây dựng prompt chứa thông tin liên quan và sử dụng LLM để sinh câu trả lời.

RAG giúp mô hình khai thác dữ liệu chuyên biệt mà không nhất thiết phải huấn luyện lại toàn bộ LLM. Tuy nhiên, chất lượng câu trả lời vẫn phụ thuộc vào dữ liệu, chất lượng truy xuất và khả năng sử dụng ngữ cảnh của mô hình.

1.4.2. Khái niệm Embedding

Embedding là phương pháp biểu diễn dữ liệu, đặc biệt là văn bản, dưới dạng vector số trong không gian nhiều chiều.

Các văn bản có nội dung hoặc ý nghĩa tương tự thường được biểu diễn bằng những vector có vị trí gần nhau theo một thước đo tương đồng nhất định. Nhờ đó, hệ thống có thể tìm kiếm tài liệu dựa trên mức độ liên quan về ngữ nghĩa, thay vì chỉ so khớp từ khóa.

Trong đề tài, mô hình paraphrase-multilingual-MiniLM-L12-v2 được sử dụng để tạo embedding cho dữ liệu kiến thức và câu hỏi của người dùng. Mô hình tạo ra vector có 384 chiều và hỗ trợ xử lý văn bản đa ngôn ngữ.

1.4.3. Tìm kiếm ngữ nghĩa và độ tương đồng vector

Tìm kiếm ngữ nghĩa (Semantic Search) là phương pháp tìm kiếm thông tin dựa trên mức độ tương đồng về ý nghĩa giữa câu truy vấn và tài liệu.

Một trong những phương pháp đo mức độ tương đồng giữa hai vector là cosine similarity, được biểu diễn như sau:

operatorname{sim}(A,B)=\frac{A\cdot B}{|A||B|}

Trong đó:

- (A) là vector biểu diễn câu hỏi.
- (B) là vector biểu diễn tài liệu.
- (A\cdot B) là tích vô hướng của hai vector.
- (|A|) và (|B|) là độ dài tương ứng của hai vector.

Khi các vector đã được chuẩn hóa về độ dài bằng 1, tích vô hướng giữa hai vector có giá trị tương đương cosine similarity.

Hệ thống có thể sử dụng điểm tương đồng để xếp hạng tài liệu và lựa chọn những nội dung có liên quan nhất đến câu hỏi.

1.4.4. Giới thiệu thư viện FAISS

FAISS (Facebook AI Similarity Search) là thư viện hỗ trợ tìm kiếm và so sánh các vector nhiều chiều, được sử dụng trong những bài toán tìm kiếm tương đồng.

FAISS cung cấp nhiều loại chỉ mục với các phương pháp tìm kiếm khác nhau, cho phép lựa chọn giải pháp phù hợp với quy mô dữ liệu và yêu cầu hiệu năng.

Trong đề tài, hệ thống sử dụng IndexFlatIP để tìm kiếm theo tích vô hướng. Các vector được chuẩn hóa trước khi lưu trữ và truy vấn, nhờ đó điểm tích vô hướng được sử dụng để đánh giá mức độ tương đồng cosine.

Bộ chỉ mục hiện tại gồm 74 vector, tương ứng với 46 tài liệu Knowledge và 28 tài liệu FAQ. Mỗi vector có 384 chiều.

1.4.5. Vai trò của RAG trong đề tài

RAG là thành phần giúp kết nối dữ liệu kiến thức về Địa đạo Củ Chi với mô hình ngôn ngữ Qwen.

Khi người dùng đặt câu hỏi, hệ thống tìm kiếm các nội dung liên quan trong bộ dữ liệu đã xây dựng. Các tài liệu được lựa chọn sẽ được đưa vào prompt để hỗ trợ LLM tạo câu trả lời.

Cách tiếp cận này có những ưu điểm như không cần huấn luyện lại LLM khi bổ sung tài liệu, cho phép sử dụng dữ liệu chuyên biệt và hỗ trợ hiển thị nguồn tham khảo.

Tuy nhiên, RAG không tự động bảo đảm rằng mọi thông tin được truy xuất đều chính xác hoặc còn hiệu lực. Đối với dữ liệu tham quan có thể thay đổi theo thời gian, việc xác minh và cập nhật nguồn thông tin vẫn cần được thực hiện.

1.5. Công nghệ và công cụ sử dụng

1.5.1. Ngôn ngữ lập trình Python

Python là ngôn ngữ lập trình bậc cao có cú pháp tương đối đơn giản, hệ sinh thái thư viện phong phú và được sử dụng rộng rãi trong phát triển phần mềm, xử lý dữ liệu và trí tuệ nhân tạo.

Trong đề tài, Python được sử dụng để xây dựng backend, triển khai cơ chế Agent, xử lý dữ liệu, thực hiện truy xuất RAG và tích hợp mô hình ngôn ngữ lớn.

1.5.2. Framework FastAPI

FastAPI là framework Python hỗ trợ xây dựng các dịch vụ web và giao diện lập trình ứng dụng (API). Framework này cung cấp khả năng định nghĩa các endpoint, xử lý dữ liệu yêu cầu và phản hồi dưới dạng JSON.

Trong đề tài, FastAPI đóng vai trò kết nối giao diện người dùng với các thành phần xử lý phía backend. Khi người dùng gửi câu hỏi, FastAPI tiếp nhận yêu cầu, gọi Agent để xử lý và trả kết quả về giao diện.

1.5.3. Hệ quản trị cơ sở dữ liệu PostgreSQL

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, hỗ trợ lưu trữ và truy vấn dữ liệu có cấu trúc.

Đề tài sử dụng PostgreSQL để quản lý phiên trò chuyện và lịch sử tin nhắn. Việc lưu trữ dữ liệu hội thoại giúp người dùng có thể xem lại các cuộc trò chuyện trước đó và hỗ trợ hệ thống khai thác ngữ cảnh khi xử lý câu hỏi tiếp theo.

Thư viện SQLAlchemy được sử dụng để định nghĩa các mô hình dữ liệu và thao tác với PostgreSQL thông qua mã nguồn Python.

1.5.4. Thư viện PyTorch và Transformers

PyTorch là thư viện hỗ trợ xây dựng và thực thi các mô hình học sâu. Transformers là thư viện cung cấp các công cụ để tải và sử dụng nhiều mô hình xử lý ngôn ngữ tự nhiên.

Trong đề tài, hai thư viện này được sử dụng để tải và chạy mô hình Qwen2.5-3B-Instruct tại máy tính cục bộ, phục vụ quá trình sinh câu trả lời.

1.5.5. Sentence Transformers và FAISS

Sentence Transformers hỗ trợ tạo biểu diễn vector cho câu và đoạn văn. Trong hệ thống, thư viện này được sử dụng cùng mô hình embedding đa ngôn ngữ để chuyển đổi câu hỏi và tài liệu thành các vector.

FAISS đảm nhiệm việc lưu trữ chỉ mục và tìm kiếm các vector tương đồng. Sự kết hợp giữa Sentence Transformers và FAISS tạo thành thành phần truy xuất ngữ nghĩa của hệ thống RAG.

1.5.6. HTML, CSS và JavaScript

HTML được sử dụng để xây dựng cấu trúc giao diện, CSS đảm nhiệm việc trình bày và định dạng, trong khi JavaScript xử lý các thao tác tương tác với người dùng.

Giao diện chatbot được xây dựng để hỗ trợ nhập câu hỏi, hiển thị phản hồi, quản lý các phiên trò chuyện và xem nguồn tham khảo. JavaScript sử dụng các yêu cầu HTTP để trao đổi dữ liệu với backend FastAPI.

CHƯƠNG II. PHÂN TÍCH VÀ XÂY DỰNG HỆ THỐNG AGENTIC CHATBOT HỖ TRỢ THAM QUAN ĐỊA ĐẠO CỦ CHI

2.1. Phân tích yêu cầu hệ thống

2.1.1. Mô tả bài toán

Địa đạo Củ Chi là địa điểm tham quan lịch sử có nhiều nội dung thông tin mà du khách cần tìm hiểu trước và trong chuyến đi. Những câu hỏi thường gặp liên quan đến lịch sử hình thành, các khu vực tham quan, giá vé, thời gian hoạt động, phương tiện di chuyển và những lưu ý khi tham quan.

Để tìm kiếm những thông tin này, người dùng thường phải truy cập các trang web hoặc đọc nhiều tài liệu khác nhau. Việc tra cứu thủ công có thể gây mất thời gian, đặc biệt khi người dùng chỉ muốn nhận câu trả lời cho một vấn đề cụ thể.

Từ nhu cầu trên, đề tài xây dựng hệ thống Agentic Chatbot hỗ trợ tham quan Địa đạo Củ Chi, cho phép người dùng đặt câu hỏi bằng ngôn ngữ tự nhiên thông qua giao diện web.

Hệ thống kết hợp mô hình ngôn ngữ lớn Qwen2.5-3B-Instruct, kỹ thuật Retrieval-Augmented Generation (RAG) và cơ chế Agent điều phối công cụ để xử lý câu hỏi. Dữ liệu kiến thức được lưu trữ và tìm kiếm thông qua FAISS, trong khi PostgreSQL được sử dụng để quản lý các phiên trò chuyện và lịch sử hội thoại.

Mục tiêu của hệ thống là hỗ trợ người dùng tiếp cận thông tin tham quan thuận tiện hơn, đồng thời xây dựng một sản phẩm thử nghiệm thể hiện sự kết hợp giữa các công nghệ trí tuệ nhân tạo và phát triển ứng dụng web.

2.1.2. Yêu cầu chức năng

Hệ thống được xây dựng với các chức năng chính như sau:

- *Chức năng tiếp nhận và trả lời câu hỏi*: Người dùng có thể nhập câu hỏi bằng ngôn ngữ tự nhiên. Hệ thống tiếp nhận yêu cầu, thực hiện các bước xử lý phù hợp và hiển thị câu trả lời trên giao diện chatbot.
- *Chức năng phân loại ý định*: Agent sử dụng các quy tắc lập trình để nhận diện một số nhóm yêu cầu như chào hỏi, câu hỏi kiến thức, câu hỏi tiếp nối, yêu cầu làm rõ, dịch thuật hoặc diễn đạt lại nội dung.

- *Chức năng truy xuất thông tin*: Đối với câu hỏi kiến thức, hệ thống tìm kiếm những tài liệu có nội dung liên quan trong bộ dữ liệu Knowledge và FAQ thông qua mô hình embedding và FAISS.
- *Chức năng sinh câu trả lời*: Hệ thống sử dụng mô hình Qwen2.5-3B-Instruct chạy cục bộ để tạo câu trả lời dựa trên câu hỏi của người dùng và thông tin tham khảo được truy xuất.
- *Chức năng quản lý phiên trò chuyện*: Người dùng có thể bắt đầu cuộc trò chuyện mới, xem danh sách các phiên trò chuyện và mở lại lịch sử trao đổi trước đó.
- *Chức năng lưu trữ lịch sử hội thoại*: Các phiên trò chuyện và tin nhắn được lưu vào cơ sở dữ liệu PostgreSQL để phục vụ việc quản lý và khai thác ngữ cảnh hội thoại.
- *Chức năng hiển thị nguồn tham khảo*: Khi hệ thống truy xuất được tài liệu liên quan, giao diện có thể hiển thị các nguồn được sử dụng trong quá trình tạo câu trả lời.
- *Chức năng hiển thị Agent Trace***:** Giao diện hỗ trợ quan sát các bước xử lý được Agent ghi nhận, bao gồm ý định, công cụ sử dụng và kết quả xử lý tương ứng.

2.1.3. Yêu cầu phi chức năng

Ngoài các chức năng chính, hệ thống cần đáp ứng một số yêu cầu phi chức năng:

- Giao diện web đơn giản, dễ sử dụng và phù hợp với thao tác hỏi đáp.
- Các thành phần phần mềm được tổ chức thành những mô-đun riêng biệt để thuận tiện cho việc quản lý và bảo trì.
- Mô hình ngôn ngữ lớn được triển khai cục bộ, không sử dụng API suy luận AI bên ngoài.
- Hệ thống có khả năng lưu trữ và truy xuất dữ liệu hội thoại bằng PostgreSQL.
- Dữ liệu kiến thức có thể được cập nhật và xây dựng lại chỉ mục FAISS khi cần thiết.
- Quá trình xử lý có thông tin ghi nhận để hỗ trợ kiểm tra và đánh giá hoạt động
- Hệ thống được triển khai ở mức thử nghiệm trên máy tính cá nhân, chưa đặt mục tiêu đáp ứng số lượng lớn người dùng đồng thời.

2.2. Phân tích tác nhân và chức năng hệ thống

2.2.1. Xác định tác nhân

Tác nhân chính của hệ thống là người dùng có nhu cầu tìm hiểu thông tin về Địa đạo Củ Chi. Người dùng tương tác với chatbot thông qua giao diện web và không cần thao tác trực tiếp với các thành phần xử lý phía backend.

Người dùng có thể thực hiện những hoạt động như gửi câu hỏi, xem câu trả lời, bắt đầu phiên trò chuyện mới, mở lại lịch sử hội thoại, xem nguồn tham khảo và theo dõi các bước xử lý của Agent.

Ngoài tác nhân người dùng, hệ thống còn có các thành phần xử lý nội bộ gồm FastAPI, Agent, RAG, mô hình ngôn ngữ và cơ sở dữ liệu. Đây là các thành phần phần mềm phối hợp để thực hiện chức năng, không phải tác nhân người dùng độc lập.

2.2.2. Các trường hợp sử dụng

Mã	Trường hợp sử dụng	Mô tả
UC01	Gửi câu hỏi	Người dùng nhập và gửi nội dung cần tra cứu.
UC02	Xem câu trả lời	Hệ thống hiển thị phản hồi sau khi xử lý.
UC03	Tạo phiên trò chuyện	Người dùng bắt đầu cuộc trò chuyện mới.
UC04	Xem lịch sử trò chuyện	Người dùng lựa chọn và mở lại phiên đã lưu.
UC05	Xem nguồn tham khảo	Người dùng xem các tài liệu được hệ thống truy xuất.
UC06	Xem Agent Trace	Người dùng xem thông tin các bước xử lý của Agent đối với câu trả lời mới.

2.2.3. Mô tả quy trình sử dụng

Khi truy cập ứng dụng, người dùng được hiển thị giao diện chatbot cùng danh sách các phiên trò chuyện đã lưu. Người dùng có thể bắt đầu một phiên mới hoặc tiếp tục phiên trò chuyện trước đó.

Sau khi nhập câu hỏi và nhấn gửi, giao diện chuyển yêu cầu đến backend để xử lý. Trong thời gian chờ, giao diện hiển thị trạng thái đang xử lý.

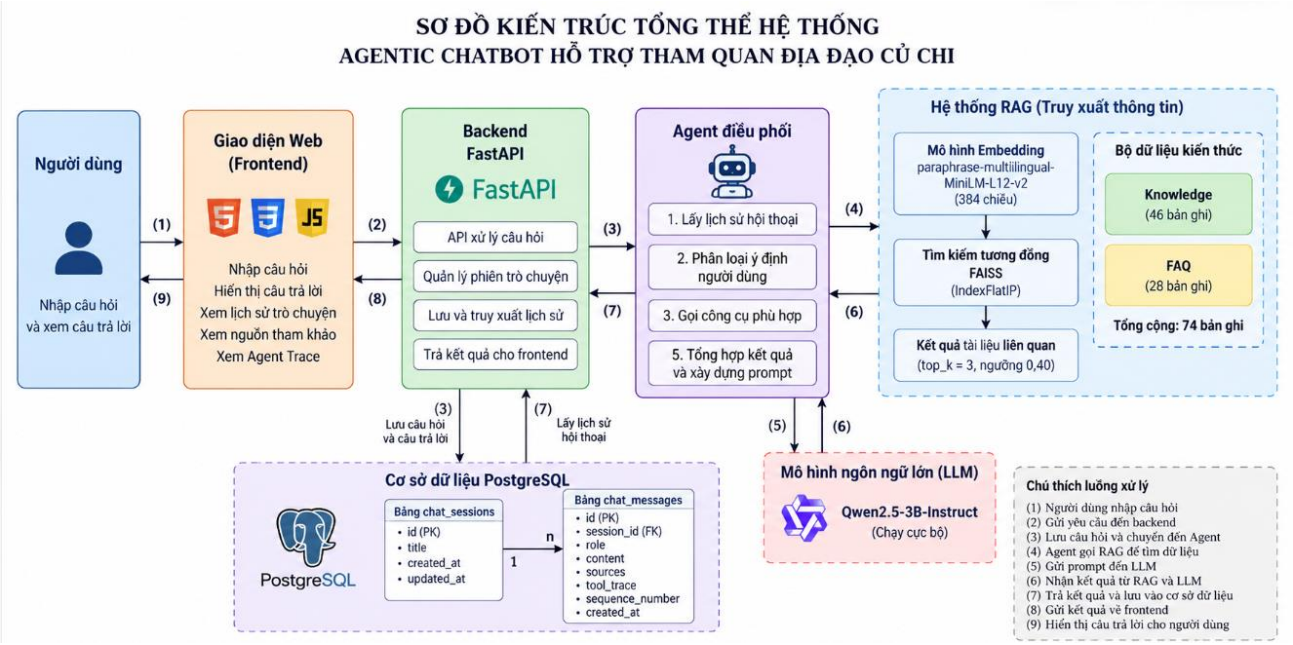
Khi nhận được kết quả từ backend, hệ thống hiển thị câu trả lời và các thông tin bổ sung như nguồn tham khảo và Agent Trace nếu có. Nội dung trao đổi được lưu trong cơ sở dữ liệu để người dùng có thể xem lại.

2.3. Xây dựng kiến trúc tổng thể của hệ thống

2.3.1. Mô hình kiến trúc

Hệ thống được xây dựng theo kiến trúc ứng dụng web kết hợp các thành phần xử lý trí tuệ nhân tạo. Các thành phần chính bao gồm:

- *Giao diện người dùng (Frontend):* Được xây dựng bằng HTML, CSS và JavaScript, có nhiệm vụ tiếp nhận thao tác của người dùng, gửi yêu cầu đến backend và hiển thị kết quả.
- *Backend FastAPI*: Tiếp nhận các yêu cầu HTTP, quản lý luồng xử lý của ứng dụng, gọi Agent và trao đổi dữ liệu với PostgreSQL.
- *Agent điều phối*: Phân loại ý định theo quy tắc, tổ chức các bước xử lý và gọi những công cụ phù hợp.
- *Hệ thống RAG*: Sử dụng mô hình embedding và FAISS để tìm kiếm các tài liệu liên quan đến câu hỏi.
- *Mô hình ngôn ngữ lớn*: Sử dụng Qwen2.5-3B-Instruct chạy cục bộ để sinh câu trả lời khi cần thiết.
- *Cơ sở dữ liệu PostgreSQL*: Lưu trữ thông tin phiên trò chuyện, tin nhắn và các dữ liệu liên quan đến quá trình hội thoại.



2.3.2. Sự phối hợp giữa các thành phần

Giao diện web đóng vai trò kết nối người dùng với hệ thống. Khi có câu hỏi mới, frontend gửi yêu cầu đến API của FastAPI.

Backend thực hiện các thao tác quản lý phiên trò chuyện và chuyển câu hỏi đến Agent. Agent xác định ý định của người dùng, sau đó thực hiện các bước xử lý tương ứng.

Đối với câu hỏi kiến thức, Agent gọi các công cụ truy xuất Knowledge và FAQ. Hệ thống RAG tạo vector biểu diễn câu hỏi, tìm kiếm tài liệu tương đồng trong FAISS và trả về các kết quả phù hợp.

Những thông tin được truy xuất sẽ được kết hợp với câu hỏi và ngữ cảnh hội thoại để xây dựng prompt. Prompt được chuyển đến Qwen2.5-3B-Instruct nhằm sinh câu trả lời.

Sau khi hoàn thành, kết quả được trả về FastAPI. Backend lưu phản hồi của chatbot vào PostgreSQL và gửi dữ liệu về frontend để hiển thị cho người dùng.

2.3.3. Đặc điểm kiến trúc

Kiến trúc được thiết kế theo hướng phân chia trách nhiệm giữa các thành phần. Mỗi mô-đun đảm nhiệm một chức năng cụ thể, giúp quá trình phát triển và kiểm thử thuận tiện hơn.

Việc tách riêng thành phần Agent, RAG, LLM và cơ sở dữ liệu cũng giúp hệ thống có khả năng điều chỉnh hoặc thay thế từng thành phần trong tương lai.

Trong phạm vi đề tài, các thành phần xử lý được triển khai trên máy tính cá nhân. Hệ thống sử dụng mô hình ngôn ngữ cục bộ, không phụ thuộc vào dịch vụ API suy luận AI bên ngoài.

2.4. Xây dựng cơ sở dữ liệu PostgreSQL

2.4.1. Mục đích sử dụng cơ sở dữ liệu

Cơ sở dữ liệu được sử dụng để lưu trữ và quản lý các phiên trò chuyện giữa người dùng với chatbot.

Việc lưu trữ lịch sử hội thoại giúp người dùng có thể mở lại các cuộc trò chuyện trước đó, đồng thời cung cấp ngữ cảnh cho Agent khi xử lý một số câu hỏi tiếp nối.

Hệ thống sử dụng PostgreSQL làm hệ quản trị cơ sở dữ liệu quan hệ và SQLAlchemy để định nghĩa các bảng, mối quan hệ và thao tác dữ liệu.

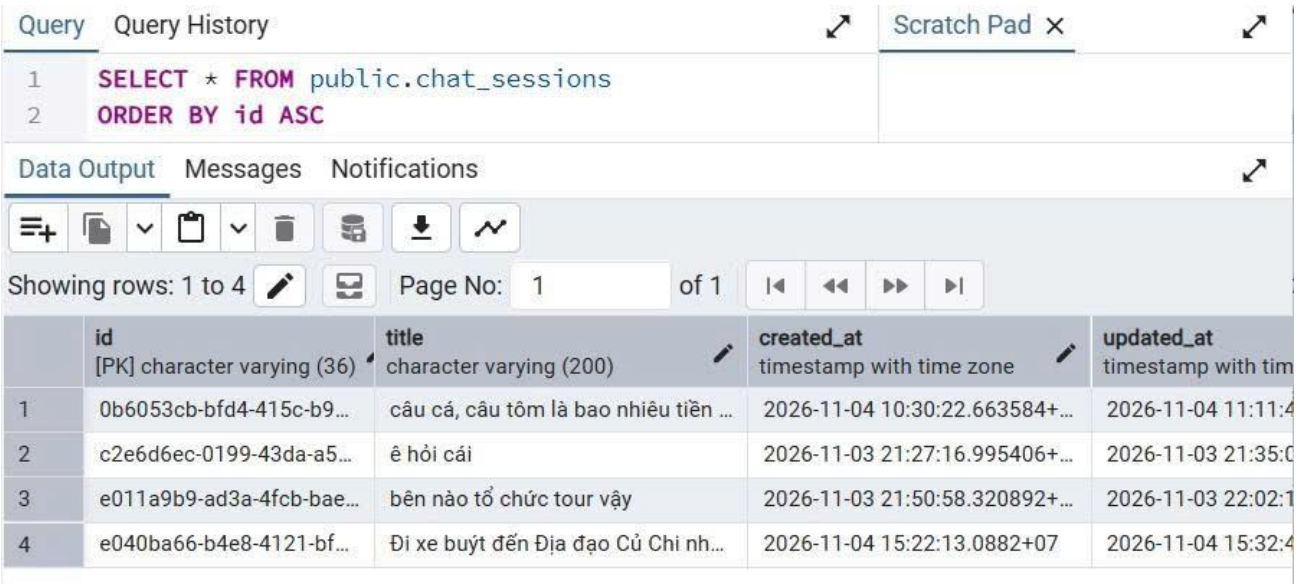
2.4.2. Thực thể phiên trò chuyện

Bảng chat_sessions được sử dụng để lưu thông tin của từng phiên trò chuyện.

Thuộc tính	Ý nghĩa

id	Mã định danh duy nhất của phiên trò chuyện.
title	Tiêu đề phiên trò chuyện.
created_at	Thời điểm tạo phiên.
updated_at	Thời điểm cập nhật phiên.

Mỗi phiên trò chuyện được định danh bằng một mã riêng. Tiêu đề phiên có thể được tạo dựa trên nội dung câu hỏi đầu tiên của người dùng.



2.4.3. Thực thể tin nhắn

Bảng chat_messages được sử dụng để lưu nội dung các tin nhắn thuộc từng phiên trò chuyện.

Thuộc tính	**Ý nghĩa**
id	Mã định danh tin nhắn.
session_id	Mã phiên trò chuyện chứa tin nhắn.
role	Vai trò của tin nhắn, chẳng hạn người dùng hoặc chatbot.
content	Nội dung tin nhắn.
sources	Thông tin nguồn tham khảo được lưu dưới dạng chuỗi JSON.
tool_trace	Thông tin các bước xử lý của Agent dưới dạng chuỗi JSON.
sequence_number	Thứ tự tin nhắn trong phiên.
created_at	Thời điểm tạo tin nhắn.

Query | Query History | Scratch Pad ×

```
SELECT * FROM public.chat_messages
ORDER BY id ASC
```

Data Output | Messages | Notifications

Showing rows: 1 to 26 | Page No: 1 of 1

	id [PK] inte	session_id character var	role character var	content text	sources text
1	1	c2e6d6ec...	user	ê hỏi cái	[null]
2	2	c2e6d6ec...	assistant	Xin chào! Tôi là trợ lý AI hỗ trợ tham ...	[null]
3	3	c2e6d6ec...	user	giá vào cổng là bao nhiêu vậy	[null]
4	4	c2e6d6ec...	assistant	Giá vé vào cổng cho người Việt Nam...	[{"id": "F003", "title": "Vé vào cổng cho n
5	5	c2e6d6ec...	user	So, how much is the entrance ticket ...	[null]
6	6	c2e6d6ec...	assistant	The entrance ticket for international ...	[{"id": "K11", "title": "Vé vào cổng – khác
7	7	c2e6d6ec...	user	giá của zipline	[null]
8	8	c2e6d6ec...	assistant	Giá vé zipline là 59.000 đồng/vé. Cầ...	[{"id": "K13", "title": "Giá vé hoạt động tra
9	9	e011a9b9...	user	bên nào tổ chức tour vậy	[null]
10	10	e011a9b9...	assistant	Joy Journeys là đơn vị tổ chức tour ...	[{"id": "K24", "title": "Joy Journeys – nhà
11	11	e011a9b9...	user	price of the swimming pool	[null]
12	12	e011a9b9...	assistant	The swimming pool costs 30,000 V...	[{"id": "F012", "title": "Có hồ bơi không?",
13	13	0b6053cb...	user	câu cá, câu tôm là bao nhiêu tiền vậy	[null]
14	14	0b6053cb...	assistant	Thông tin về giá vé câu cá, câu tôm t...	[{"id": "K14", "title": "Giá bắn súng thể th

Query | Query History | Scratch Pad ×

```
SELECT * FROM public.chat_messages
ORDER BY id ASC
```

Data Output | Messages | Notifications

Showing rows: 1 to 26 | Page No: 1 of 1

		tool_trace text	sequence_number integer	created_at timestamp with time zone
1		[null]	1	2026-11-03 21:27:17.016412+...
2		{"user_query": "ê hỏi cái", "det...	2	2026-11-03 21:27:17.044405+...
3		[null]	3	2026-11-03 21:27:32.237424+...
4	": "K10", "title": "Vé vào c...	{"user_query": "giá vào cổng l...	4	2026-11-03 21:29:09.365238+...
5		[null]	5	2026-11-03 21:30:00.9737+07
6	"K10", "title": "Vé vào cổ...	{"user_query": "So, how much...	6	2026-11-03 21:30:59.592679+...
7		[null]	7	2026-11-03 21:34:21.638222+...
8		{"user_query": "giá của zipline...	8	2026-11-03 21:35:08.188064+...
9		[null]	1	2026-11-03 21:50:58.342826+...
10	e": "Có hướng dẫn viên ti...	{"user_query": "bên nào tổ ch...	2	2026-11-03 21:52:30.526895+...
11		[null]	3	2026-11-03 22:01:21.890886+...
12	u trú", "score": 0.479}]	{"user_query": "price of the s...	4	2026-11-03 22:02:19.355118+...
13		[null]	1	2026-11-04 10:30:22.703407+...
14		{"user_query": "câu cá, câu tô...	2	2026-11-04 10:32:38.544741+...

2.4.4. Mối quan hệ giữa các thực thể

Hai bảng chat_sessions và chat_messages có quan hệ một–nhiều.

Một phiên trò chuyện có thể chứa nhiều tin nhắn, trong khi mỗi tin nhắn chỉ thuộc về một phiên trò chuyện.

Khóa ngoại session_id trong bảng chat_messages tham chiếu đến khóa chính id của bảng chat_sessions.

Mối quan hệ này giúp tổ chức lịch sử hội thoại theo từng phiên. Khi một phiên trò chuyện bị xóa thông qua cơ chế quản lý dữ liệu của ứng dụng, các tin nhắn liên quan cũng được xử lý theo cấu hình quan hệ đã thiết lập.

2.4.5. Vai trò của PostgreSQL trong hệ thống

PostgreSQL không được sử dụng để lưu trữ các vector embedding của hệ thống RAG. Các vector và chỉ mục tìm kiếm được lưu riêng thông qua FAISS.

Trong kiến trúc đề tài, PostgreSQL chủ yếu đảm nhiệm việc quản lý phiên chat và lịch sử tin nhắn, còn FAISS đảm nhiệm việc tìm kiếm thông tin trong bộ dữ liệu kiến thức.

Việc phân chia này giúp mỗi thành phần lưu trữ được sử dụng đúng với mục đích.

2.5. Xây dựng bộ dữ liệu và hệ thống RAG

2.5.1. Xây dựng bộ dữ liệu kiến thức

Bộ dữ liệu của hệ thống được tổ chức thành hai nhóm chính:

- **Knowledge:** Chứa những thông tin về lịch sử, địa điểm tham quan, hoạt động trải nghiệm, phương tiện di chuyển và các nội dung liên quan đến Địa đạo Củ Chi.
- **FAQ:** Chứa các câu hỏi thường gặp cùng nội dung trả lời tương ứng, phục vụ việc tra cứu những vấn đề phổ biến của du khách.

Bộ dữ liệu mở rộng được sử dụng để xây dựng chỉ mục gồm 46 bản ghi Knowledge và 28 bản ghi FAQ, tổng cộng 74 bản ghi.

Nhóm dữ liệu	**Số bản ghi**
Knowledge	46
FAQ	28
Tổng cộng	74

Dữ liệu được lưu dưới dạng các tệp CSV và được chương trình đọc để xây dựng chỉ mục tìm kiếm. Ngoài ra, dự án còn có tệp Excel và JSONL phục vụ việc tổ chức, lưu trữ dữ liệu ở các định dạng khác nhau.

2.5.2. Tiền xử lý và tạo embedding

Trước khi thực hiện tìm kiếm ngữ nghĩa, các bản ghi được chuyển thành nội dung văn bản phù hợp để tạo embedding.

Hệ thống sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2 thông qua thư viện Sentence Transformers để chuyển đổi văn bản thành vector có 384 chiều.

Mỗi tài liệu được biểu diễn bằng một vector, đồng thời giữ lại thông tin mô tả như mã tài liệu, loại tài liệu, tiêu đề và nội dung để phục vụ việc hiển thị nguồn tham khảo. Các vector được chuẩn hóa trước khi đưa vào FAISS nhằm hỗ trợ tính toán độ tương đồng.

2.5.3. Xây dựng chỉ mục FAISS

Hệ thống sử dụng FAISS với loại chỉ mục IndexFlatIP để thực hiện tìm kiếm dựa trên tích vô hướng giữa các vector.

Sau khi các tài liệu được chuyển thành embedding, hệ thống lưu chỉ mục vector vào tệp index.faiss. Thông tin tài liệu tương ứng được lưu trong tệp documents.pkl.

Kết quả kiểm tra bộ chỉ mục hiện tại cho thấy có 74 vector với 384 chiều, tương ứng với 74 bản ghi dữ liệu đã xây dựng.

Việc lưu chỉ mục vào tệp cho phép hệ thống tải lại dữ liệu tìm kiếm trong những lần khởi động tiếp theo mà không cần tạo embedding cho toàn bộ tài liệu mỗi lần chạy.

2.5.4. Quy trình truy xuất tài liệu

Khi nhận được câu hỏi kiến thức, hệ thống RAG thực hiện các bước:

1. Tiếp nhận nội dung câu hỏi từ Agent.
2. Sử dụng mô hình embedding để chuyển câu hỏi thành vector.
3. Chuẩn hóa vector câu hỏi.
4. Tìm kiếm các vector tương đồng trong chỉ mục FAISS.
5. Lọc kết quả dựa trên ngưỡng điểm tương đồng được cấu hình.
6. Trả về những tài liệu phù hợp cùng thông tin nguồn tham khảo.

Trong cấu hình hiện tại, hệ thống sử dụng ngưỡng tương đồng 0,40 và tham số số lượng kết quả tìm kiếm top_k = 3.

Agent có các công cụ riêng để tìm kiếm nhóm Knowledge và nhóm FAQ. Đối với câu hỏi kiến thức thông thường, Agent thực hiện truy xuất cả hai nhóm dữ liệu, sau đó tổng hợp các kết quả phù hợp.

2.5.5. Vai trò của RAG đối với chất lượng câu trả lời

RAG giúp mô hình ngôn ngữ có thêm thông tin tham khảo liên quan đến câu hỏi của người dùng.

Thay vì chỉ sử dụng kiến thức đã được học trong quá trình huấn luyện, Qwen được cung cấp nội dung tài liệu truy xuất từ bộ dữ liệu Địa đạo Củ Chi.

Cách tiếp cận này giúp hệ thống tập trung vào phạm vi kiến thức của đề tài và hỗ trợ việc hiển thị nguồn tham khảo.

Tuy nhiên, chất lượng phản hồi vẫn phụ thuộc vào nội dung tài liệu và khả năng xử lý của LLM. Một số thông tin như giá vé, thời gian hoạt động hoặc thông tin liên hệ có thể thay đổi theo thời gian, vì vậy cần được kiểm tra và cập nhật khi sử dụng trong thực tế.

2.6. Xây dựng Agent và cơ chế điều phối công cụ

2.6.1. Vai trò của Agent

Agent là thành phần trung tâm điều phối quá trình xử lý yêu cầu của người dùng.

Khi nhận được câu hỏi từ backend, Agent xác định ý định của người dùng, lựa chọn nhánh xử lý phù hợp và phối hợp các công cụ cần thiết để tạo phản hồi.

Trong đề tài, Agent được xây dựng bằng Python và sử dụng cơ chế phân loại ý định dựa trên quy tắc. Điều này có nghĩa là việc lựa chọn hướng xử lý chủ yếu dựa trên các điều kiện được lập trình trước, không phải do LLM tự quyết định toàn bộ quy trình.

2.6.2. Phân loại ý định người dùng

Hệ thống định nghĩa các nhóm ý định như chào hỏi, hỏi kiến thức, hỏi tiếp nối, yêu cầu làm rõ, dịch thuật, diễn đạt lại nội dung và một số nhóm hỗ trợ khác.

Hàm phân loại ý định sử dụng nội dung câu hỏi và ngữ cảnh hội thoại để xác định nhánh xử lý.

Ví dụ, khi người dùng nhập lời chào, Agent có thể tạo phản hồi trực tiếp mà không cần thực hiện tìm kiếm RAG hoặc gọi LLM.

Đối với câu hỏi về thông tin tham quan, Agent chuyển sang nhánh truy xuất dữ liệu và sinh câu trả lời.

Một số loại ý định đã được định nghĩa trong mã nguồn nhưng chưa có đầy đủ điều kiện nhận diện tương ứng. Vì vậy, khả năng phân loại hiện tại phụ thuộc vào các quy tắc đã triển khai.

2.6.3. Các công cụ của Agent

Agent sử dụng ba công cụ chính:

- *Công cụ tìm kiếm Knowledge*: Thực hiện tìm kiếm các tài liệu kiến thức liên quan đến câu hỏi của người dùng thông qua hệ thống RAG.
- *Công cụ tìm kiếm FAQ*: Tìm kiếm các câu hỏi thường gặp có nội dung tương đồng với yêu cầu hiện tại.
- *Công cụ lấy lịch sử hội thoại*: Truy xuất các tin nhắn gần đây trong phiên trò chuyện từ PostgreSQL nhằm cung cấp ngữ cảnh cho quá trình xử lý.

Việc sử dụng các công cụ giúp Agent kết nối giữa yêu cầu của người dùng với dữ liệu kiến thức và thông tin hội thoại.

2.6.4. Cơ chế điều phối xử lý

Quy trình điều phối được thực hiện theo logic lập trình trong agent.py.

Đầu tiên, Agent tiếp nhận câu hỏi và thông tin phiên trò chuyện. Sau đó, hệ thống khai thác lịch sử hội thoại và phân loại ý định.

Tùy theo ý định được xác định, Agent thực hiện nhánh xử lý phù hợp. Với các yêu cầu đơn giản như chào hỏi hoặc làm rõ nội dung, hệ thống có thể tạo phản hồi trực tiếp.

Đối với câu hỏi kiến thức và câu hỏi tiếp nối, Agent gọi các công cụ tìm kiếm Knowledge và FAQ, tổng hợp kết quả, xây dựng prompt và chuyển đến LLM để sinh câu trả lời.

Kết quả xử lý được trả về cùng thông tin nguồn tham khảo và Agent Trace.

2.6.5. Ghi nhận Agent Trace

Agent Trace là thông tin ghi nhận quá trình xử lý yêu cầu, hỗ trợ việc quan sát và kiểm tra hoạt động của hệ thống.

Các thông tin được ghi nhận có thể bao gồm ý định được phân loại, các bước xử lý, công cụ đã sử dụng, kết quả truy xuất và thời gian thực hiện.

Trong quá trình thử nghiệm, Agent Trace giúp xác định câu hỏi đã được xử lý theo nhánh nào, có sử dụng công cụ truy xuất dữ liệu hay không và tổng thời gian xử lý là bao nhiêu.

Cần phân biệt Agent Trace với cơ chế tự lập kế hoạch. Việc hệ thống ghi nhận nhiều bước xử lý không đồng nghĩa với việc LLM đã tự xây dựng và thực hiện một kế hoạch độc lập.

2.7. Luồng xử lý câu hỏi và sinh câu trả lời

2.7.1. Các bước xử lý câu hỏi

Đây là quy trình trung tâm của hệ thống, thể hiện sự phối hợp giữa giao diện web, FastAPI, PostgreSQL, Agent, RAG và mô hình ngôn ngữ Qwen2.5-3B-Instruct.

- **Bước 1 – Người dùng nhập câu hỏi**

Người dùng truy cập giao diện chatbot trên trình duyệt và nhập nội dung câu hỏi vào ô trò chuyện.

Ví dụ: "Đi xe buýt đến Địa đạo Củ Chi như thế nào?"

Sau khi người dùng nhấn nút gửi, JavaScript tiếp nhận nội dung và gửi yêu cầu HTTP đến backend thông qua endpoint POST /api/chat.

Yêu cầu chứa nội dung câu hỏi và mã phiên trò chuyện nếu người dùng đang tiếp tục một phiên đã có.

- **Bước 2 – FastAPI tiếp nhận và lưu câu hỏi**

Backend FastAPI tiếp nhận yêu cầu từ giao diện.

Nếu người dùng chưa có phiên trò chuyện, hệ thống tạo phiên mới và sinh mã định danh cho phiên đó.

Sau đó, nội dung câu hỏi được lưu vào bảng chat_messages trong PostgreSQL với vai trò người dùng.

Backend tiếp tục chuyển câu hỏi và mã phiên trò chuyện đến Agent để thực hiện xử lý.

- **Bước 3 – Agent tiếp nhận và phân loại ý định**

Agent tiếp nhận câu hỏi, lấy thông tin lịch sử hội thoại gần đây và xác định ý định của người dùng bằng các quy tắc đã lập trình.

Với câu hỏi "Đi xe buýt đến Địa đạo Củ Chi như thế nào?", hệ thống nhận diện đây là yêu cầu tra cứu kiến thức liên quan đến phương tiện di chuyển.

Câu hỏi được chuyển sang nhánh xử lý kiến thức để thực hiện truy xuất dữ liệu.

- **Bước 4 – Agent gọi công cụ tìm kiếm**

Đối với câu hỏi kiến thức, Agent thực hiện tìm kiếm trong cả hai nhóm dữ liệu Knowledge và FAQ.

Các công cụ tìm kiếm gửi nội dung câu hỏi đến thành phần RAG.

RAG sử dụng mô hình embedding để chuyển câu hỏi thành vector, sau đó so sánh với các vector đã lưu trong FAISS.

Những tài liệu có điểm tương đồng đáp ứng ngưỡng cấu hình được lựa chọn và trả về cho Agent.

Ví dụ, với câu hỏi về xe buýt, kết quả truy xuất có thể chứa thông tin liên quan đến phương tiện công cộng và cách di chuyển đến Địa đạo Củ Chi.

- **Bước 5 – Tổng hợp tài liệu và xây dựng prompt**

Sau khi nhận kết quả truy xuất, Agent tổng hợp và lựa chọn các tài liệu phù hợp để làm ngữ cảnh cho mô hình ngôn ngữ.

Prompt được xây dựng từ các thành phần chính gồm:

- Hướng dẫn cách trả lời dành cho mô hình.
- Câu hỏi hiện tại của người dùng.
- Nội dung các tài liệu được truy xuất.
- Ngữ cảnh hội thoại có liên quan.

Việc bổ sung tài liệu vào prompt giúp mô hình có cơ sở thông tin cụ thể về Địa đạo Củ Chi khi tạo câu trả lời.

- **Bước 6 – Qwen sinh câu trả lời**

Prompt được chuyển đến mô hình Qwen2.5-3B-Instruct đang chạy cục bộ.

Mô hình xử lý nội dung đầu vào và lần lượt sinh các token để tạo thành câu trả lời bằng ngôn ngữ tự nhiên.

Trong quá trình này, mô hình sử dụng thông tin được cung cấp trong prompt để phản hồi câu hỏi của người dùng.

Thời gian sinh câu trả lời phụ thuộc vào độ dài nội dung đầu ra, cấu hình suy luận và tài nguyên phần cứng của máy tính.

- **Bước 7 – Agent tổng hợp kết quả xử lý**

Sau khi nhận được nội dung do LLM tạo ra, Agent tổng hợp kết quả cuối cùng.

Thông tin trả về bao gồm:

- Nội dung câu trả lời.
- Danh sách nguồn tham khảo được sử dụng.
- Agent Trace ghi nhận các bước xử lý.

Kết quả được chuyển về backend FastAPI.

- **Bước 8 – Lưu câu trả lời vào PostgreSQL**

Backend lưu câu trả lời của chatbot vào bảng chat_messages với vai trò tương ứng. Các thông tin về nguồn tham khảo và Agent Trace cũng được lưu cùng bản ghi tin nhắn.

Việc lưu trữ này giúp hệ thống duy trì lịch sử trao đổi và hỗ trợ truy xuất dữ liệu hội thoại trong những lần sử dụng tiếp theo.

- **Bước 9 – Hiển thị câu trả lời trên giao diện**

Sau khi hoàn thành xử lý và lưu dữ liệu, FastAPI trả kết quả về frontend dưới dạng JSON.

JavaScript tiếp nhận phản hồi và cập nhật giao diện chatbot.

Người dùng nhìn thấy nội dung câu trả lời, các nguồn tham khảo liên quan và có thể mở phần Agent Trace để quan sát quá trình xử lý.

Như vậy, một yêu cầu hỏi đáp được xử lý qua chuỗi thành phần từ giao diện web đến backend, Agent, RAG, LLM và cơ sở dữ liệu trước khi kết quả được hiển thị.

2.7.2. Xử lý các yêu cầu không cần truy xuất RAG

Không phải mọi yêu cầu đều phải thực hiện đầy đủ quy trình truy xuất dữ liệu và sinh văn bản.

Đối với một số yêu cầu đơn giản như lời chào hoặc yêu cầu làm rõ, Agent có thể tạo phản hồi trực tiếp theo quy tắc đã lập trình.

Các yêu cầu như dịch thuật hoặc diễn đạt lại nội dung trước đó có thể sử dụng ngữ cảnh hội thoại và mô hình ngôn ngữ mà không cần thực hiện quy trình tìm kiếm Knowledge và FAQ như câu hỏi kiến thức thông thường.

Việc phân chia nhánh xử lý giúp hệ thống lựa chọn quy trình phù hợp với từng nhóm yêu cầu.

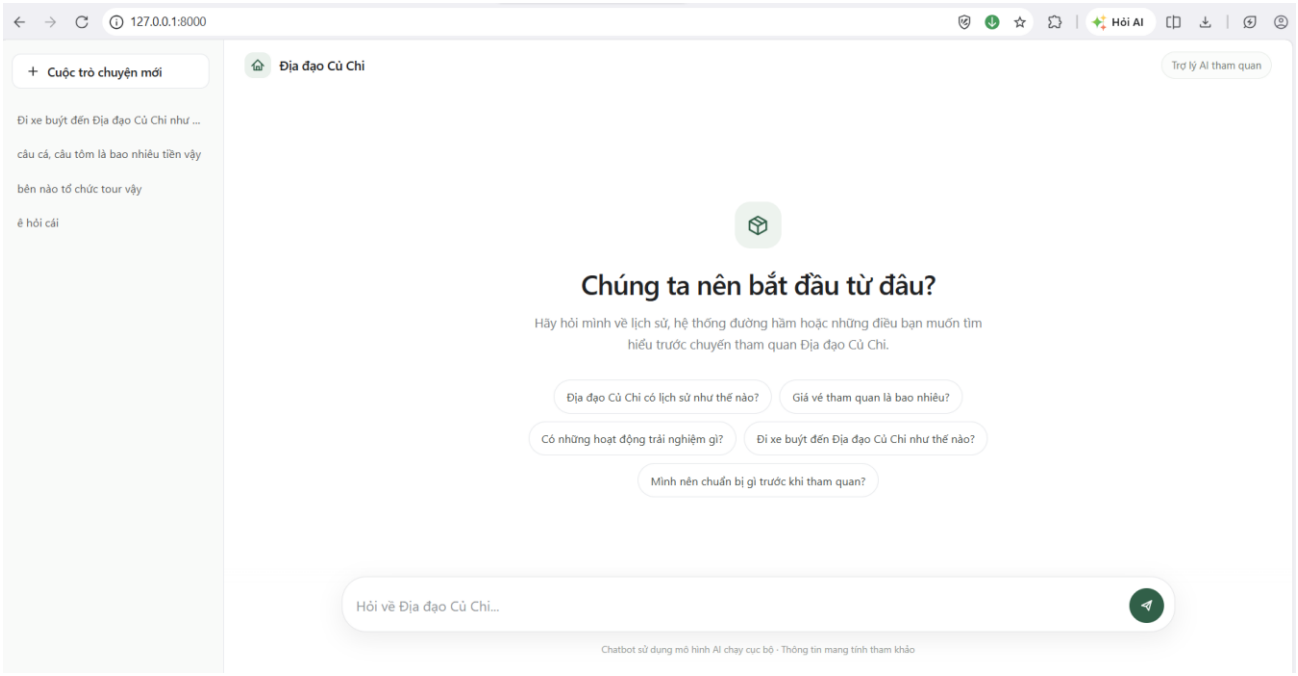
2.8. Thiết kế giao diện và tích hợp hệ thống

2.8.1. Giao diện chatbot

Giao diện được xây dựng bằng HTML, CSS và JavaScript, tổ chức theo dạng ứng dụng trò chuyện trên trình duyệt.

Khu vực chính hiển thị nội dung hội thoại giữa người dùng và chatbot. Người dùng nhập câu hỏi vào ô soạn tin nhắn và gửi yêu cầu để nhận phản hồi.

Giao diện còn có khu vực quản lý phiên trò chuyện, cho phép bắt đầu phiên mới và mở lại các phiên đã được lưu trữ.



2.8.2. Hiển thị trạng thái xử lý

Do mô hình ngôn ngữ chạy cục bộ có thể cần nhiều thời gian để sinh câu trả lời, giao diện được thiết kế để hiển thị trạng thái chờ trong quá trình xử lý.

Việc hiển thị trạng thái giúp người dùng nhận biết hệ thống đang thực hiện yêu cầu thay vì cho rằng ứng dụng đã ngừng hoạt động.

Sau khi backend trả kết quả, giao diện cập nhật nội dung phản hồi tương ứng.

2.8.3. Hiển thị nguồn tham khảo và Agent Trace

Đối với những câu hỏi có sử dụng dữ liệu RAG, giao diện có thể hiển thị danh sách nguồn tham khảo liên quan đến câu trả lời.

Ngoài ra, chức năng Agent Trace cho phép người dùng xem thông tin các bước xử lý được ghi nhận trong lượt hỏi đáp mới.

Những thông tin này hỗ trợ việc kiểm tra hoạt động của hệ thống, đặc biệt trong quá trình thử nghiệm và trình bày đồ án.

2.8.4. Tích hợp frontend và backend

Frontend trao đổi dữ liệu với backend thông qua các API do FastAPI cung cấp.

Các API chính bao gồm:

API	Phương thức	Chức năng
/	GET	Trả về giao diện chatbot.
/health	GET	Kiểm tra trạng thái các thành phần hệ thống.
/api/chat	POST	Tiếp nhận câu hỏi và trả kết quả xử lý.
/api/sessions	GET	Lấy danh sách các phiên trò chuyện.
/api/sessions	POST	Tạo phiên trò chuyện mới.
/api/sessions/{session_id}/messages	GET	Lấy lịch sử tin nhắn của một phiên.
/api/sessions/{session_id}	DELETE	Xóa phiên trò chuyện thông qua API.

Trong phiên bản giao diện hiện tại, chức năng xóa phiên trò chuyện đã được xây dựng ở backend nhưng chưa có nút thao tác tương ứng trên giao diện.

2.8.5. Đánh giá quá trình tích hợp

Quá trình tích hợp cho phép các thành phần giao diện, backend, Agent, RAG, LLM và PostgreSQL phối hợp trong một luồng xử lý thống nhất.

Người dùng không cần thao tác trực tiếp với mô hình AI hoặc cơ sở dữ liệu mà có thể sử dụng các chức năng thông qua giao diện web.

Việc kết hợp các thành phần tạo thành một ứng dụng chatbot có khả năng tiếp nhận câu hỏi, truy xuất kiến thức, sinh phản hồi và lưu trữ lịch sử hội thoại.

Đặc biệt, chương đã mô tả chi tiết quy trình xử lý từ khi người dùng nhập câu hỏi, gửi yêu cầu đến FastAPI, Agent phân loại ý định, RAG tìm kiếm tài liệu, Qwen sinh câu trả lời, cho đến khi kết quả được lưu trữ và hiển thị trên giao diện.

Trên cơ sở hệ thống đã xây dựng, Chương 3 sẽ trình bày môi trường triển khai, các kịch bản thử nghiệm, kết quả hoạt động thực tế và những hạn chế cần tiếp tục cải thiện.

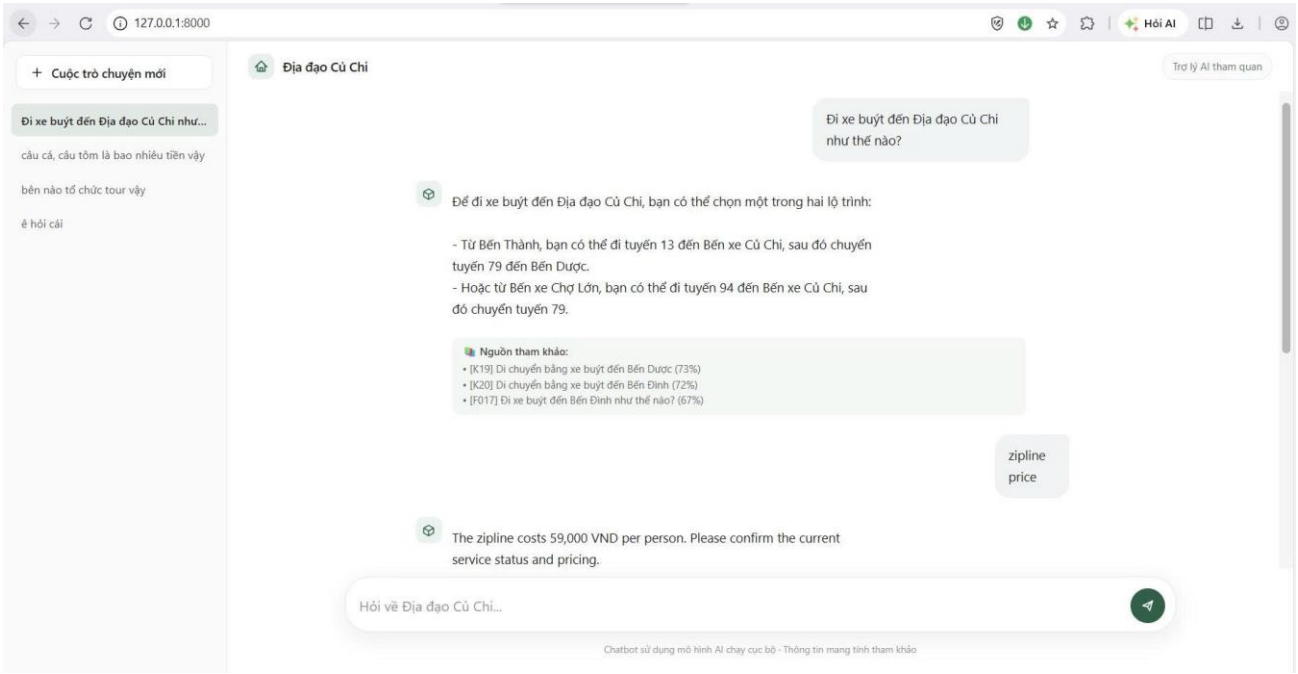
CHƯƠNG III. KẾT QUẢ THỰC NGHIỆM

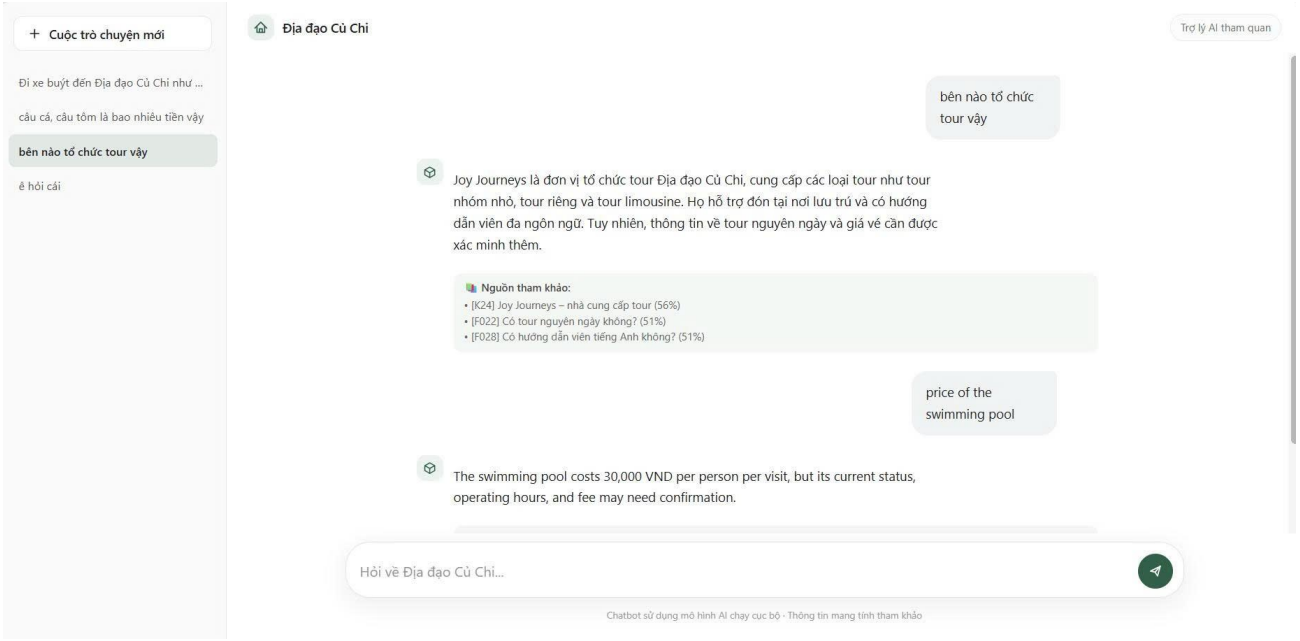
3.1. Kết quả hoạt động của hệ thống

Sau quá trình xây dựng và tích hợp, hệ thống đã thực hiện được các chức năng chính:

- Tiếp nhận câu hỏi và hiển thị câu trả lời trên giao diện web.
- Phân loại ý định người dùng và điều phối các công cụ xử lý thông qua Agent.
- Truy xuất thông tin liên quan từ bộ dữ liệu Knowledge và FAQ bằng RAG.
- Sử dụng mô hình Qwen2.5-3B-Instruct chạy cục bộ để sinh câu trả lời.
- Lưu trữ phiên trò chuyện và lịch sử tin nhắn trong PostgreSQL.
- Hiển thị nguồn tham khảo và các bước xử lý thông qua Agent Trace.

Trong quá trình thử nghiệm, hệ thống đã tiếp nhận và xử lý các câu hỏi liên quan đến phương tiện di chuyển, giá vé và hoạt động tham quan Địa đạo Củ Chi.





3.2. Đánh giá kết quả đạt được

Qua quá trình xây dựng và thử nghiệm, đề tài đã đạt được mục tiêu cơ bản là phát triển một ứng dụng chatbot hỗ trợ tra cứu thông tin về Địa đạo Củ Chi.

Hệ thống đã tích hợp được các thành phần FastAPI, Agent, RAG, FAISS, Qwen2.5-3B-Instruct và PostgreSQL thành một quy trình xử lý thống nhất. Người dùng có thể đặt câu hỏi thông qua giao diện web và nhận câu trả lời từ mô hình ngôn ngữ dựa trên dữ liệu được truy xuất.

Bên cạnh đó, hệ thống đã xây dựng được chức năng lưu trữ lịch sử hội thoại, hiển thị nguồn tham khảo và ghi nhận các bước xử lý của Agent.

Tuy nhiên, hệ thống vẫn còn một số hạn chế. Tốc độ sinh câu trả lời còn chậm, bộ dữ liệu chưa bao phủ toàn bộ nhu cầu tra cứu của du khách và một số thông tin cần được xác minh, cập nhật. Ngoài ra, Agent hiện điều phối công cụ theo quy tắc, chưa có khả năng tự lập kế hoạch độc lập bằng LLM.

3.3. Kết luận

Sau quá trình nghiên cứu và thực hiện đề tài "Xây dựng Agentic – Chatbot hỗ trợ tham quan Địa đạo Củ Chi", hệ thống đã được xây dựng và thử nghiệm với các chức năng cơ bản của một ứng dụng chatbot hỗ trợ tra cứu thông tin.

Đề tài đã kết hợp kỹ thuật RAG, mô hình ngôn ngữ Qwen2.5-3B-Instruct và cơ chế Agent điều phối công cụ để xử lý câu hỏi của người dùng. Đồng thời, hệ thống sử dụng PostgreSQL để lưu trữ lịch sử hội thoại và xây dựng giao diện web để người dùng tương tác.

Tuy nhiên, hệ thống vẫn còn hạn chế về tốc độ xử lý, phạm vi dữ liệu và khả năng điều phối của Agent. Đây là những vấn đề cần tiếp tục nghiên cứu và cải thiện.

3.4. Hướng phát triển

Trong tương lai, hệ thống có thể được phát triển theo các hướng sau:

- Tối ưu mô hình ngôn ngữ: Nghiên cứu các phương pháp giảm dung lượng và tăng tốc độ suy luận nhằm rút ngắn thời gian phản hồi.
- Mở rộng và cập nhật dữ liệu: Bổ sung thông tin tham quan, kiểm chứng các nội dung quan trọng và cập nhật những dữ liệu có thể thay đổi như giá vé, thời gian hoạt động và thông tin liên hệ.
- Nâng cao khả năng của Agent: Hoàn thiện cơ chế phân loại ý định và nghiên cứu khả năng lập kế hoạch, lựa chọn công cụ linh hoạt hơn.
- Cải thiện giao diện: Hoàn thiện các chức năng quản lý lịch sử trò chuyện và hiển thị đầy đủ thông tin Agent Trace.
- Mở rộng kiểm thử: Xây dựng bộ câu hỏi đánh giá để kiểm tra chất lượng truy xuất, độ chính xác của câu trả lời và hiệu năng hệ thống.

Tài liệu tham khảo

- Dữ liệu

https://www.vietfuntravel.com.vn/blog/cac-tro-choi-tai-di-tich-dia-dao-cu-chi.html

https://vinwonders.com/vi/wonderpedia/news/dia-dao-cu-chi-khu-di-tich-duoi-long-dat-o-sai-gon/

http://diadaocuchi.com.vn/gioi-thieu-ve-khu-di-tich-lich-su-dia-dao-cu-chi-37.html

- Trang web tham khảo

https://www.chatlab.com/industry/museums/

https://www.alimentarium.org/en/basic-page/alima

- Bài báo

https://www.mdpi.com/2504-2289/7/3/148

https://www.mdpi.com/2076-3417/15/17/9398

https://arxiv.org/abs/2005.11401